

21A – M1 Party, Tenancy & Security: Implementation Guide

Everything the M1 developer needs to build the module: what it is, what you inherit, the schema as runnable DDL, seed data, package layout, the API slice, every command as a handler specification, queries and caches, screens, tests and the tickets in build order. Self-contained: what it needs from other documents is restated here and marked.

Field	Value
Document	21A – implementation guide for 21 M1 Party, Tenancy & Security v0.1
Version	v0.1 – for the M1 developer; Sprint 1 (from 21 September 2026, after Sprint 0)
Date	8 September 2026
Prerequisite	Sprint 0 complete: 17A repository and hello module, 19A kernel services (scope, identity, permissions, audit, events, config, scheduling). M1 replaces nothing in the kernel; it fills the party and security schemas the kernel already reads.
Estimated effort	About 17 developer-days for one full-stack developer (Section 10)
Restated content	Marked <i>restated from doc X §Y</i> ; the design decisions remain in 21, 18, 19, 30; this guide is the build.

Read this first. M1 is small in code and large in consequence: every other module resolves scope, permissions and relationships through what you build. Build it in the order of Section 10, keep the contract in Section 5 unchanged once M2 starts designing against it (week 2 gate), and raise anything you would have to guess as a question in the Section 11 list rather than deciding it silently.

1. What you are building

Restated from doc 21 §1. M1 answers four questions for the system: who exists (entities, locations, till positions, devices), who trades with whom on what terms (relationships), who may do what (users, roles, permissions, external grants), and what each caller may see (scope and policy class). It owns reference data and policy; the kernel enforces the policy on every request and database session.

You build	You do not build
Schemas party and security with their tables, constraints, RLS policies and seed data	Membership, shares, governance records
22 commands, 10 queries, 8 event families (Section 5)	Password self-service, SSO with external providers
The identity-provider calls behind CreateUser, ResetCredential, DeactivateUser through the kernel wrapper	The provider itself, token issuance, step-up (kernel)
Nine web screens (Section 8)	Any till screen (the till consumes the operator snapshot the kernel builds from your ListOperators)
BulkRegister with a validation report	Catalogue cleansing (M2), device provisioning (doc 31)

Where M1 sits: layer 1, depending on the kernel only; M2–M9 call its api and query packages.

2. What you inherit from Sprint 0

From	You use	How
17A kernel.api	ScopeContext, @CommandHandler, Handles, ProblemException, Ids.next(), DomainEvent	Every handler implements Handles; throw ProblemException with a message id for guard failures
19A §1 scope	Session variables set per transaction; kernel.scope_entity() etc. in RLS	Your policies use the 17A §6.3 template unchanged
19A §2 identity	IdentityProviderClient: createUser, disableUser, setTemporaryPassword, resetTotp, revokeSessions; PinHasher	Call inside your handlers; the wrapper enforces the scope rule, you enforce yours first
19A §3 permissions	PermissionResolver, Sod, Grant helpers, TillSnapshotSigner	Your role handlers call the guardrail helpers; the resolver reads your tables
19A §4 audit	AuditFacade.record(type, subject, before, after, scope, reason, witness)	One call per command, inside the transaction
19A §5 events	EventPublisher.publish(event)	One or more per command, inside the transaction; types in Section 5.3
19A §11 configuration	ConfigRegistry typed getters	PIN policy, idle locks, MFA windows, tier flag (Section 7)
19A §7 numbering	NumberingService.registerSeries / closeSeries / holderChange	Called from your location, position and device handlers
19A §8 sync	SnapshotContributor interface; SnapshotChangeLog.record	You contribute operators and permissions (Section 7.3)
17A §7 web shell	ScopeBanner, DocumentHeader, ApprovalBar, ReasonCapture, StateChip, Timeline, ExceptionQueue; generated client; useT	Section 8
17A §12 hello module	The handler shape, test shape, migration shape	Run make new-module NAME=m1party SCHEMA=party then add the security schema by hand

3. Schema

Restated from doc 18 part A v0.2 and doc 21 §3. Two migration locations: db/migration/m1party/ for schema party and db/migration/m1security/ for security. The full files are in the repository; the fragments below show every table with the constraints that carry design rules. Common columns created_at, updated_at, created_by are omitted for brevity.

3.1 party schema

```
-- V0001__party.sql
CREATE TABLE party.entity (
  entity_id uuid PRIMARY KEY, entity_code varchar(12) NOT NULL UNIQUE,
  entity_type text NOT NULL CHECK (entity_type IN ('FEDERATION','DISTRIBUTOR','MPCS')),
  legal_name_en text NOT NULL COLLATE en_icu, legal_name_si text COLLATE si_icu, legal_name_ta text COLLATE
    ta_icu,
  registration_no varchar(40), vat_registration_no varchar(20), district varchar(40),
  financial_year_start_month smallint NOT NULL DEFAULT 1 CHECK (financial_year_start_month BETWEEN 1 AND 12),
  default_language text NOT NULL DEFAULT 'en' CHECK (default_language IN ('en','si','ta')),
  costing_policy text NOT NULL DEFAULT 'WEIGHTED_AVERAGE', credit_exposure_warn_only boolean NOT NULL DEFAULT
    true,
  responsible_officer_user_id uuid, data_governance_signed_on date,
  status text NOT NULL DEFAULT 'ONBOARDING' CHECK (status IN ('ONBOARDING','ACTIVE','SUSPENDED')),
  owner_entity_id uuid GENERATED ALWAYS AS (entity_id) STORED -- P1: an entity owns itself
);
CREATE UNIQUE INDEX one_federation ON party.entity ((entity_type)) WHERE entity_type = 'FEDERATION';

CREATE TABLE party.entity_relationship (
  relationship_id uuid PRIMARY KEY, seller_entity_id uuid NOT NULL, buyer_entity_id uuid NOT NULL,
  price_list_id uuid, credit_limit numeric(14,2), payment_terms_days smallint,
  discrepancy_window_days smallint NOT NULL DEFAULT 7, order_lock_hours_before_eta smallint NOT NULL DEFAULT
    24,
  allocation_rule text NOT NULL DEFAULT 'FCFS',
  status text NOT NULL DEFAULT 'DRAFT' CHECK (status IN ('DRAFT','ACTIVE','SUSPENDED')),
  effective_from date NOT NULL, effective_to date,
  owner_entity_id uuid GENERATED ALWAYS AS (seller_entity_id) STORED,
  CHECK (seller_entity_id <> buyer_entity_id),
  CHECK (effective_to IS NULL OR effective_to >= effective_from),
  EXCLUDE USING gist (seller_entity_id WITH =, buyer_entity_id WITH =,
    daterange(effective_from, coalesce(effective_to, 'infinity'::date), '[]') WITH &&)
  WHERE (status = 'ACTIVE') -- A-I3: one ACTIVE per pair per date
);

CREATE TABLE party.location (
  location_id uuid PRIMARY KEY, owner_entity_id uuid NOT NULL, location_code varchar(12) NOT NULL,
  location_type text NOT NULL CHECK (location_type IN ('WAREHOUSE','SHOP','OFFICE')),
  name_en text NOT NULL COLLATE en_icu, name_si text COLLATE si_icu, name_ta text COLLATE ta_icu,
  address text, district varchar(40), geo_lat numeric(9,6), geo_lng numeric(9,6),
  language text CHECK (language IN ('en','si','ta')), trading_hours jsonb,
  size_band text CHECK (size_band IN ('S','M','L')), connectivity_spec_met boolean NOT NULL DEFAULT false,
  primary_till_position_id uuid,
  status text NOT NULL DEFAULT 'PLANNED' CHECK (status IN ('PLANNED','ONBOARDING','ACTIVE','DORMANT')),
  UNIQUE (owner_entity_id, location_code)
);
-- A-I2: owner immutable
CREATE OR REPLACE FUNCTION party.location_owner_immutable() RETURNS trigger LANGUAGE plpgsql AS $$
BEGIN IF NEW.owner_entity_id <> OLD.owner_entity_id THEN RAISE EXCEPTION 'location.owner_immutable'; END IF;
RETURN NEW; END $$;
CREATE TRIGGER trg_location_owner BEFORE UPDATE ON party.location FOR EACH ROW EXECUTE FUNCTION
  party.location_owner_immutable();

CREATE TABLE party.till_position (
  till_position_id uuid PRIMARY KEY, location_id uuid NOT NULL REFERENCES party.location, position_no smallint
    NOT NULL,
  status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE','RETIRED')),
  owner_entity_id uuid NOT NULL, UNIQUE (location_id, position_no)
);
CREATE TABLE party.device (
  device_id uuid PRIMARY KEY, hardware_serial varchar(64) NOT NULL UNIQUE,
  device_kind text NOT NULL CHECK (device_kind IN ('POS_TERMINAL','WORKSTATION','DRIVER_MOBILE')),
  owner_entity_id uuid NOT NULL, current_till_position_id uuid REFERENCES party.till_position,
  enrolled_at timestamptz, last_seen_at timestamptz, app_version varchar(20),
  status text NOT NULL DEFAULT 'ENROLLED' CHECK (status IN ('ENROLLED','ACTIVE','SUSPENDED','RETIRED'))
);
CREATE UNIQUE INDEX one_device_per_position ON party.device (current_till_position_id) WHERE
  current_till_position_id IS NOT NULL;
```

```
-- RLS on every table: template from 17A 6.3 (own_read, own_write, fed_view, ext_view); grants:
GRANT SELECT, INSERT, UPDATE ON ALL TABLES IN SCHEMA party TO app_rw; -- reference data may UPDATE; no
DELETE
```

3.2 security schema

```
-- V0001__security.sql
CREATE TABLE security.app_user (
  user_id uuid PRIMARY KEY, home_entity_id uuid NOT NULL, username varchar(64) NOT NULL UNIQUE,
  display_name text NOT NULL, language text NOT NULL DEFAULT 'en',
  user_kind text NOT NULL CHECK (user_kind IN ('BACK_OFFICE', 'TILL', 'BOTH', 'EXTERNAL')),
  provider_subject text UNIQUE, pin_hash text, pin_changed_at timestamptz, pin_history jsonb NOT NULL DEFAULT
  '[]',
  sandbox boolean NOT NULL DEFAULT false, succeeds_user_id uuid,
  status text NOT NULL DEFAULT 'PENDING' CHECK (status IN ('PENDING', 'ACTIVE', 'LOCKED', 'DEACTIVATED')),
  owner_entity_id uuid GENERATED ALWAYS AS (home_entity_id) STORED
);
CREATE TABLE security.permission (
  permission_code varchar(64) PRIMARY KEY, module varchar(16) NOT NULL, description_en text NOT NULL,
  offline_allowed boolean NOT NULL DEFAULT false, requires_mfa boolean NOT NULL DEFAULT false,
  scope text NOT NULL CHECK (scope IN ('LOCATION', 'ENTITY', 'FEDERATION')), limits_schema jsonb
);
CREATE TABLE security.permission_catalogue_version (rv int PRIMARY KEY, published_at timestamptz NOT NULL
  DEFAULT now());
CREATE TABLE security.role (
  role_id uuid PRIMARY KEY, owner_entity_id uuid, -- NULL = federation template
  name_en text NOT NULL, name_si text, name_ta text, is_template boolean NOT NULL DEFAULT false,
  role_class text NOT NULL DEFAULT 'OWN' CHECK (role_class IN ('OWN', 'FEDERATION_VIEW', 'EXTERNAL_TIMEBOXED')),
  template_role_id uuid REFERENCES security.role, template_version_seen int, version int NOT NULL DEFAULT 1,
  status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN ('ACTIVE', 'RETIRED')),
  UNIQUE (owner_entity_id, name_en)
);
CREATE TABLE security.role_permission (
  role_id uuid NOT NULL REFERENCES security.role, permission_code varchar(64) NOT NULL REFERENCES
  security.permission,
  limits jsonb, PRIMARY KEY (role_id, permission_code));
CREATE TABLE security.user_role (
  user_id uuid NOT NULL REFERENCES security.app_user, role_id uuid NOT NULL REFERENCES security.role,
  scope_entity_id uuid NOT NULL, scope_location_id uuid,
  PRIMARY KEY (user_id, role_id, scope_entity_id, coalesce(scope_location_id,
  '00000000-0000-0000-0000-000000000000'::uuid));
CREATE TABLE security.sod_pair (
  sod_pair_id uuid PRIMARY KEY, permission_a varchar(64) NOT NULL, permission_b varchar(64) NOT NULL,
  mode text NOT NULL CHECK (mode IN ('INSTANCE', 'ROLE')), owner_entity_id uuid,
  CHECK (permission_a < permission_b), UNIQUE (permission_a, permission_b, owner_entity_id));
CREATE TABLE security.external_grant (
  grant_id uuid PRIMARY KEY, grantee_user_id uuid NOT NULL REFERENCES security.app_user,
  scope_entity_ids uuid[] NOT NULL, valid_from timestamptz NOT NULL, valid_until timestamptz NOT NULL,
  reason text NOT NULL, status text NOT NULL DEFAULT 'ACTIVE' CHECK (status IN
  ('ACTIVE', 'EXPIRED', 'REVOKED')),
  owner_entity_id uuid NOT NULL, -- the Federation
  CHECK (valid_until <= valid_from + interval '12 months'));
-- RLS: app_user, role, user_role by owner_entity_id (an MPCs admin also reads its shops' users: policy adds
-- owner_entity_id = kernel.scope_entity() OR home entity's parent = scope for shop-scoped users - see 19A);
-- permission, permission_catalogue_version, template roles (owner NULL): readable by all authenticated
scopes.
```

3.3 Seed data

Files under seed/m1/, loaded by the kernel SeedLoader on start (upsert by key; never changes rows an entity has edited).

```
# seed/m1/permissions.yaml (excerpt; full file has ~120 codes across modules; M1 owns the catalogue file)
- code: gov.entity.register      module: m1  scope: FEDERATION  mfa: false offline: false
- code: gov.entity.activate      module: m1  scope: FEDERATION  mfa: true
- code: gov.user.manage          module: m1  scope: ENTITY      mfa: true
- code: gov.role.manage          module: m1  scope: ENTITY      mfa: true
- code: gov.external.grant       module: m1  scope: FEDERATION  mfa: true
- code: gov.audit.review         module: m1  scope: ENTITY
- code: prt.relationship.open     module: m1  scope: ENTITY
- code: prt.relationship.activate module: m1  scope: ENTITY
- code: prt.relationship.amend    module: m1  scope: ENTITY
- code: prt.relationship.suspend module: m1  scope: ENTITY      mfa: true
- code: bil.creditlimit.change   module: m4  scope: ENTITY      mfa: true # owned by M4, granted here
- code: prt.location.register    module: m1  scope: ENTITY
- code: prt.location.activate    module: m1  scope: ENTITY
- code: prt.location.primary     module: m1  scope: ENTITY      mfa: true
- code: prt.position.manage      module: m1  scope: ENTITY
```

```

- code: sys.device.enrol      module: m1  scope: ENTITY    mfa: true
- code: sys.device.suspend    module: m1  scope: ENTITY    mfa: true
- code: rpt.analyst.query     module: m8  scope: FEDERATION    # doc 20 Appendix A
# seed/m1/role-templates.yaml (excerpt of the 17 templates: 14 of doc 07 + shop-in-charge, driver, regulator,
  auditor)
- name: shop-in-charge  class: OWN
  permissions: [pos.receipt.issue, pos.receipt.void, pos.refund.same_session, pos.negative_stock.acknowledge,
    pos.mrp.pick, shop.grn.confirm, shop.count.record, inv.writeoff.request, pos.session.manage]
- name: mpcs-accountant  class: OWN
  permissions: [inv.writeoff.approve {limits: {max_value: 25000}}, inv.adjust.approve {limits: {max_value:
    50000}},
    bil.invoice.issue, bil.creditnote.issue, bil.payment.record, cus.account.manage]
- name: federation-view  class: FEDERATION_VIEW  permissions: []      # read everything, write nothing
- name: regulator        class: EXTERNAL_TIMEBOXED permissions: []
# seed/m1/sod-pairs.yaml
- [inv.writeoff.approve, inv.writeoff.request, INSTANCE]
- [inv.writeoff.request, inv.writeoff.witness, INSTANCE]
- [inv.adjust.approve, inv.adjust.request, INSTANCE]
- [gov.audit.review, gov.user.manage, INSTANCE]      # self-review guard

```

4. Package layout

```
lk.coopfed.knoweb.m1party/                                     @ApplicationModule(allowedDependencies = {"kernel"})
  api/                  commands (22 records), events (8 families), PartyApi facade, value types (EntityCode, Scope,
                        Terms)
  query/                PartyQueries, SecurityQueries (interfaces only; records, never entities)
  internal/
    entity/             Entity, EntityRepository, RegisterEntityHandler, ActivateEntityHandler, ...
    relationship/        Relationship, RelationshipRepository, OpenTradingRelationshipHandler,
                        AmendRelationshipTermsHandler, ...
    location/            Location, TillPosition, Device, handlers, SeriesHooks (calls NumberingService)
    security/            AppUser, Role, RolePermission, UserRole, SodPair, ExternalGrant, handlers, Guardrails
    snapshot/            OperatorSnapshotContributor, PermissionSnapshotContributor (implement kernel
                        SnapshotContributor)
    bulk/                BulkRegisterHandler, RowValidators, ValidationReport
    queries/             PartyQueriesImpl, SecurityQueriesImpl
  web/                  EntitiesController, RelationshipsController, LocationsController, DevicesController,
                        UsersController, RolesController, GrantsController, BulkController (implement generated
                        m1party.yaml)
```

One Modulith module for both schemas: M1 is one contract and one developer; the two schemas are an ownership convenience for RLS grants, not a module boundary.

5. API slice: openapi/m1party.yaml

Restated from doc 21 §5. Paths under /v1/party and /v1/security; every mutating operation requires the Idempotency-Key header; errors are problem details whose type is a message id. Full file in the repository; the excerpt shows the conventions.

```
openapi: 3.1.0
paths:
  /v1/party/entities:
    post: { operationId: registerEntity, x-permission: gov.entity.register, x-scope: FEDERATION,
            requestBody: RegisterEntityRequest, responses: { "201": EntityCreated, "409": Problem } }
    get:  { operationId: listEntities, parameters: [status, district, cursor, limit], responses: { "200":
            EntityPage } }
  /v1/party/entities/{entityId}/activate:
    post: { operationId: activateEntity, x-permission: gov.entity.activate, x-mfa: true, responses: { "204":
            {} } }
  /v1/party/relationships:
    post: { operationId: openTradingRelationship, x-permission: prt.relationship.open }
  /v1/party/relationships/{relationshipId}/amend:
    post: { operationId: amendRelationshipTerms, x-permission: prt.relationship.amend,
            description: "credit_limit change additionally requires bil.creditlimit.change with MFA" }
  /v1/party/locations/{locationId}/primary-till:
    put: { operationId: setPrimaryTill, x-permission: prt.location.primary, x-mfa: true }
  /v1/party/devices/{deviceId}/assign:
    post: { operationId: assignDeviceToPosition, x-permission: sys.device.enrol }
  /v1/security/users:
    post: createUser (gov.user.manage, mfa) get: listUsers
  /v1/security/users/{userId}/reset-credential:
    post: resetCredential (gov.user.manage, mfa)
  /v1/security/roles:
    post: createRole get: listRoles
  /v1/security/roles/{roleId}/diff:
    get: getRoleDiff
  /v1/security/assignments:
    post: assignRole delete: revokeRole
  /v1/security/external-grants:
    post: grantExternalView (gov.external.grant, mfa) get:
        listExternalGrants
  /v1/party/bulk-register:
    post: bulkRegister (multipart; returns ValidationReport)
components:
  schemas:
    RegisterEntityRequest: { entityCode, entityType, legalNameEn, legalNameSi?, legalNameTa?, registrationNo,
                            vatRegistrationNo, district, defaultLanguage, financialYearStartMonth }
    AmendTermsRequest:    { effectiveFrom, priceListId?, creditLimit?, paymentTermsDays?,
                            discrepancyWindowDays?,
                            orderLockHoursBeforeEta?, allocationRule?, reasonCode, reasonText? }
    Problem:              { type: "relationship.overlap", title, status, detail, instance, params }
# example - 409 on overlapping relationship
{ "type": "relationship.overlap", "status": 409,
  "params": { "existingRelationshipId": "018f...", "effectiveFrom": "2026-10-01" } }
```

The x-permission, x-mfa and x-scope extensions are read by a build-time check that compares them with the @CommandHandler annotations, so the API document and the code cannot disagree.

6. Handler specifications

Every handler follows the hello shape (17A §12): the kernel interceptor has already checked permission, MFA and idempotency; the handler runs guards in the listed order, mutates, records audit, publishes events, in one transaction. Guard failures throw `ProblemException(messageId, params)`.

Command	Guards (in order)	Mutation	Audit type	Events
RegisterEntity	caller FEDERATION; code unique; type valid; exactly-one-federation	insert ONBOARDING	ENTITY_REGISTERED	entity.registered.v1
ActivateEntity	status ONBOARDING; officer set; ≥1 user with gov.user.manage; VAT no present	status ACTIVE	ENTITY_ACTIVATED	entity.activated.v1
SuspendEntity / Reinstatement	status transition valid; reason present	status	ENTITY_SUSPENDED / REINSTATED	entity.suspended/reinstated.v1
AppointResponsible Officer	user belongs to entity; caller FEDERATION or entity admin	officer fields	ENTITY_UPDATE	entity.updated.v1
OpenTradingRelationship	seller = caller scope; seller ≠ buyer; both ACTIVE or ONBOARDING; tier rule or config flag; no overlapping ACTIVE (pre-check; DB exclusion is the backstop)	insert DRAFT	RELATIONSHIP_OPENED	relationship.opened.v1
ActivateRelationship	status DRAFT; M3 query: price list belongs to seller and is published	status ACTIVE	RELATIONSHIP_ACTIVATED	relationship.activated.v1
AmendRelationshipTerms	status ACTIVE; effective_from > current row's effective_from; if credit_limit changes: caller holds bil.creditlimit.change and MFA fresh (interceptor checks the extra permission via requiresAlso); reason present	close current row (effective_to = effective_from - 1 day); insert new ACTIVE row	RELATIONSHIP_AMENDED; CREDIT_LIMIT_CHANGED when applicable	relationship.amended.v1; credit_limit.changed.v1
SuspendRelationship	ACTIVE; reason	status	RELATIONSHIP_SUSPENDED	relationship.suspended.v1
RegisterLocation	owner = caller scope; code unique in owner; type valid	insert PLANNED	LOCATION_REGISTERED	location.registered.v1
StartOnboarding / ActivateLocation / MarkDormant / Reactivate	transition valid; ACTIVE requires connectivity_spec_met and (SHOP) primary till and ≥1 operator	status	LOCATION_*	location.*.v1
SetPrimaryTill	position belongs to location; position ACTIVE; reason	update flag; NumberingService.holderChange(locationSeries, newDevice)	LOCATION_PRIMARY_CHANGED	location.primary_changed.v1 (kernel emits series.holder_changed.v1)
RegisterTillPosition	location SHOP; position_no unique	insert; NumberingService.registerSeries(TILL_POSITION, ...)	POSITION_REGISTERED	till_position.registered.v1
RetireTillPosition	no device assigned; no open session (M6 query)	status RETIRED; closeSeries	POSITION_RETIRED	till_position.retired.v1
EnrolDevice	serial unique; staging record attested (doc 31)	insert ENROLLED	DEVICE_ENROLLED	device.enrolled.v1
AssignDeviceToPosition	device ENROLLED or ACTIVE; position free; app_version ≥ floor; on re-assign: old device outbox drained (kernel SyncStatus.drained(oldDevice)) or loss recorded	set position; status ACTIVE; kernel counter transfer	DEVICE_ASSIGNED / POSITION_CHANGED	device.assigned/position_changed.v1
SuspendDevice / Reinstatement / Retire	transition valid; reason; retire requires unassigned	status; unassign on suspend keeps position for return	DEVICE_*	device.*.v1 (gateway turns suspended into a revoke)
CreateUser	home entity within caller scope (own entity, or shop-scoped for MPCs admin); username unique; kind valid	insert PENDING; idp.createUser; for TILL/BOTH: no PIN yet	USER_CREATED	user.created.v1

Command	Guards (in order)	Mutation	Audit type	Events
ResetCredential	user in scope; not DEACTIVATED	BACK_OFFICE: idp.setTemporaryPassword; TILL: new PIN validated against policy and history, PinHasher, pin_history rotate	USER_CREDENTIAL_RESET	user.credential_reset.v1
DeactivateUser	in scope; not the last gov.user.manage holder in the entity (Grant.lastAdminGuard); reason	status DEACTIVATED; idp.disableUser; revokeSessions; user_role rows kept	USER_DEACTIVATED	user.deactivated.v1
CreateRole / AmendRole	owner = caller scope (or FEDERATION for templates); Grant.withinGrantor; no FEDERATION permission unless template; Sod.roleModeConflict; limits validate against each permission's limits_schema	insert / replace role_permission set; version + 1	ROLE_CHANGE with before/after permission sets	role.changed.v1
RetireRole	no active assignments	status RETIRED	ROLE_CHANGE	role.changed.v1
AssignRole / RevokeRole	role owner = scope entity or template; scope within caller's scope; user in scope; role class permits	insert / delete user_role (the one DELETE M1 performs; audited)	ROLE_ASSIGNED / REVOKED	role.assigned/revoked.v1
SetSodPairMode	both permissions exist; entity scope	upsert sod_pair	SOD_PAIR_CHANGED	sod_pair.changed.v1
GrantExternalView / Revoke	caller FEDERATION; grantee kind EXTERNAL; valid_until ≤ 12 months; reason	insert / status	EXTERNAL_GRANTED_ISSUED / REVOKED	external_grant.issued/revoked.v1
BulkRegister	per row, the guards of the row's command	each valid row through its handler in its own transaction; report collects outcomes	per row	per row

6.1 Pseudocode for the three non-obvious handlers

```
// AmendRelationshipTermsHandler.handle(cmd, ctx)
Relationship cur = repo.activeFor(cmd.relationshipId()).orElseThrow("relationship.not_active");
require(cur.sellerEntityId().equals(ctx.entityId()), "relationship.not_seller");
require(cmd.effectiveFrom().isAfter(cur.effectiveFrom()), "relationship.effective_from_not_after");
boolean limitChange = cmd.creditLimit() != null && !cmd.creditLimit().equals(cur.creditLimit());
if (limitChange) { perms.require(ctx, "bil.creditlimit.change"); mfa.requireFresh(ctx,
    "bil.creditlimit.change"); }
require(cmd.reasonCode() != null, "reason.required");
Relationship next = cur.copyWith(cmd.terms()).withId(Ids.next()).effective(cmd.effectiveFrom(), null);
cur.close(cmd.effectiveFrom().minusDays(1)); // effective_to on the old row; never edit its terms
repo.save(cur); repo.save(next); // exclusion constraint backstops overlap
audit.record(RELATIONSHIP_AMENDED, Subject.of(next), cur.terms(), next.terms(), ctx, cmd.reason());
if (limitChange) audit.record(CREDIT_LIMIT_CHANGED, Subject.of(next), cur.creditLimit(), next.creditLimit(),
    ctx, cmd.reason());
events.publish(new RelationshipAmended(next.id(), cur.id(), next.effectiveFrom()));
if (limitChange) events.publish(new CreditLimitChanged(next.id(), cur.creditLimit(), next.creditLimit()));

// AssignDeviceToPositionHandler.handle(cmd, ctx)
Device d = repo.device(cmd.deviceId()); TillPosition p = repo.position(cmd.positionId());
require(d.ownerEntityId().equals(ctx.entityId()) && p.ownerEntityId().equals(ctx.entityId()),
    "scope.mismatch");
require(p.isActive() && repo.deviceAt(p.id()).isEmpty(), "position.occupied");
require(versions.atOrAboveFloor(d.appVersion()), "device.below_floor");
Optional<Device> old = repo.deviceAt(p.id()); // re-assign path when the position was previously held
if (cmd.previousDeviceId() != null) {
    require(sync.drained(cmd.previousDeviceId()) || cmd.lossRecorded(), "device.outbox_not_drained");
    mfa.requireFresh(ctx, "sys.device.enrol");
}
d.assign(p.id()); repo.save(d);
numbering.holderChange(p.seriesIds(), d.id()); // kernel: counter transfer, audited
series.holder_changed
if (p.isPrimaryForLocation()) numbering.holderChange(location.seriesIds(), d.id());
audit.record(cmd.previousDeviceId()==null ? DEVICE_ASSIGNED : DEVICE_POSITION_CHANGED, Subject.of(d), old, d,
    ctx, cmd.reason());
events.publish(new DeviceAssigned(d.id(), p.id(), cmd.previousDeviceId()));

// AmendRoleHandler.handle(cmd, ctx) -- guardrails first, then the set replace
Role r = repo.role(cmd.roleId()); require(r.ownerEntityId().equals(ctx.entityId()) || (r.isTemplate() &&
```

```

    ctx.isFederation()), "role.not_owner");
Set<Perm> wanted = cmd.permissions();
grant.withinGrantor(ctx, wanted); // "permission.not_held" listing the offenders
require(r.isTemplate() || wanted.stream().noneMatch(Perm::isFederationScope), "permission.federation_only");
sod.roleModeConflict(wanted, ctx.entityId()); // "sod.role_conflict" naming the pair
limits.validate(wanted); // each limit against permission.limits_schema
Set<Perm> before = r.permissions(); r.replacePermissions(wanted); r.bumpVersion(); repo.save(r);
audit.record(ROLE_CHANGED, Subject.of(r), before, wanted, ctx, null);
events.publish(new RoleChanged(r.id(), r.ownerEntityId(), r.version(), diff(before, wanted)));

```

7. Queries, caches and snapshot contributions

Query	Implementation	Scope and cache
ResolveScope(user)	app_user + user_role joined to role.role_class; external grants for EXTERNAL users	Kernel-only; the kernel caches by user for the token lifetime
ResolvePermissions(user, rv)	user_role → role_permission → permission, union per scope with limits	Kernel PermissionCache keyed (user, rv); you invalidate by publishing the role/assignment events
LookupRelationship(seller, buyer, date)	index on (seller, buyer, effective_from)	PARTY or FEDERATION_VIEW; cached 60 s per instance
ListRelationships(entity, role)	as seller or buyer	OWN
GetEntity / ListEntities	cursor-paged; PARTY sees names only via a projection record	OWN, PARTY, FEDERATION_VIEW
ListLocations / GetLocation	with primary till and hours	OWN; PARTY sees ship-to names for drops
ListOperators(location)	users with a TILL/BOTH kind holding an assignment scoped to the location or its entity, with pin_hash and offline permissions	Kernel snapshot builder only (never exposed on the API)
ListDevices(entity)	join last_seen from the device row (heartbeat updates it through the kernel)	OWN, FEDERATION_VIEW
ListRoles / GetRoleDiff	diff computed against the template's current permission set	OWN
ListExternalGrants	active and expired	FEDERATION

7.3 Snapshot contributors (doc 32 §5.1)

```
// OperatorSnapshotContributor implements SnapshotContributor
// tables(): ["operator"]
// rowsFor(locationId, rowIds): ListOperators(locationId) -> {userId, displayName, language, pinHash,
//   permissions[], rv}
// signed by the kernel TillSnapshotSigner (19A 3)
// PermissionSnapshotContributor: tables(): ["permission"] -> the offline_allowed subset of the catalogue with
//   rv
// Change-log producers (in your handlers, via the kernel):
//   user.created/credential_reset/deactivated -> SnapshotChangeLog.record(locations of the user's scopes,
//     "operator", userId, op)
//   role.changed/assigned/revoked           -> ... "operator" for affected users (urgent when a permission
//     is removed)
//   permission catalogue version bump       -> all locations, "permission" (worker fan-out)
```

8. Screens

Restated from doc 21 §8 and doc 30 §5.1. Nine web screens on the shell components; no till screens.

Wireframes are given as zone sketches; the design-tool prototypes (doc 30 DR-3) refine them but must keep the zones.

```
+-----+
| ScopeBanner: COOPFED (Federation) | Language: en | fed-admin |
+-----+
| Society register [ Register society ] [ Bulk upload ] |
| Filter: status [All v] district [All v] search [_____] |
| Code Name Type District Status Officer Shops (active/all) |
| M042 Kandy South MPCS MPCS Kandy ONBOARDING - 0 / 3 > |
| D007 Central Distributors DISTRIBUTOR Kandy ACTIVE set 1 / 1 > |
+-----+
Society card (detail): DocumentHeader-style card: code, names (three), registration, VAT, district
Tabs: Overview | Relationships | Shops | Staff | Duties | Devices | History (Timeline of audit events)
Onboarding checklist tile: [x] admin user [ ] responsible officer [x] VAT no [ ] shops active (0/3)
ApprovalBar: [Activate] (disabled: reason "responsible officer not named") [Suspend...]

Agreement sheet (relationship detail):
Header: seller -> buyer, status chip, effective range
Terms form (read-only when ACTIVE; "Amend terms" opens a new effective-dated form):
  Price list [Central 2026-Q4 v] Credit limit [Rs 5,000,000] Terms [30] days
  Discrepancy window [7] days Lock window [24] h before ETA Allocation [FCFS v]
Timeline: 2026-04-01 opened by ... | 2026-07-01 amended (credit limit 3,000,000 -> 5,000,000, reason: ...)
ApprovalBar: [Amend terms] [Suspend] -> credit limit change: ReasonCapture then StepUpDialog

Duties list (role builder):
```

Left: role list for the entity (cloned templates marked, drift badge "template updated: 2 changes")

Right: permissions grouped by paper document:

Receipt ☒ issue ☒ void ☒ refund same session ☐ refund cross session

GRN ☒ confirm Count ☒ record

Write-off ☒ request ☐ witness ☐ approve (limit Rs [_____])

Conflicts shown inline: "approve + request: allowed (instance rule) - the same person cannot do both on one document"

ApprovalBar: [Save role] -> StepUpDialog

Screen	Components	Message ids (prefix m1.)	Notes
Society register / card	ScopeBanner, list, DocumentHeader card, Timeline, ApprovalBar, ReasonCapture	entity.*	Activate disabled with reason text from the checklist
Agreement sheet	DocumentHeader, form, Timeline, ApprovalBar, ReasonCapture, StepUpDialog	relationship.*	Effective-dated history; never edits the current row
Shop list / shop card	list, card, hours editor, primary-till selector with ReasonCapture	location.*	Connectivity gate checkbox is a fact, not a toggle: it records who confirmed
Staff register	list, form, StepUpDialog	user.*	Roles shown as duty names; scope as shop names
Duties list	two-pane, grouped checklist, limits inputs, diff banner	role.*	Section 8 sketch
Asset register	list, card, ApprovalBar (suspend, reassign with ReasonCapture)	device.*	Reassign warns when the old device is not drained
Access letter	form, list	grant.*	Expiry shown prominently
Bulk registration	upload, ValidationReport table with per-row reasons, commit	bulk.*	Reasons are message ids from the handlers' problems
Exception queue	shared ExceptionQueue	—	Filtered to governance event types

9. Tests

Layer	Cases (fixtures in src/test/resources/m1/)	Target
Unit (handlers with in-memory repos)	Every guard in Section 6 as a failing case with its message id; the three pseudocode handlers fully; effective-date closure arithmetic incl. month ends	100% of guards
Contract	Generated client against a running app: every operation; idempotent resend returns identical body; MFA-gated operations return step_up_required when stale; x-permission/x-mfa match annotations (build check)	Every operation
State machine	All transitions of doc 21 §4 (six machines); illegal transitions rejected	Every transition
Property-based (jqwik)	Random role edits never produce a ROLE-mode conflict; random assignments never exceed grantor scope; random effective ranges never violate the exclusion constraint	1000 cases each
RLS matrix (integration)	Rows for party.* and security.*: distributor user vs MPCS rows; shop-scoped user vs sibling shop operators; Federation view reads all, writes nothing; expired grant reads nothing; NONE scope reads nothing	Full matrix
Scenario (integration)	Flows 6.1–6.6 of doc 21: onboarding an MPCS with three shops via BulkRegister; opening and amending a relationship; building and assigning a role; regulator grant lifecycle; device replacement with counter transfer (asserts the kernel series.holder_changed event)	Six flows
Performance	ResolvePermissions under 2 ms from cache; BulkRegister of 40 shops, 120 users, 60 devices under 60 s with report	Two targets
Web	Vitest for the duties list diff and conflict rendering; Playwright: register society → activate blocked → appoint officer → activate; amend credit limit with step-up	Two flows

10. Tickets in build order

#	Ticket	Est. (days)	Done when
M1-01	Scaffold from hello; party and security migrations with RLS and grants; schema-ownership check green	1.5	Migrations apply; RLS matrix skeleton runs
M1-02	Seed files: permission catalogue, templates, SoD pairs, config items; SeedLoader wiring	1	Catalogue visible via ResolvePermissions
M1-03	Entities: aggregate, repository, 4 handlers, queries, controller, tests	1.5	Register → activate flow
M1-04	Relationships: aggregate, 4 handlers incl. AmendTerms, M3 query stub, exclusion tests	2	Scenario 6.2 and 6.4
M1-05	Locations, positions: handlers, NumberingService hooks, primary till	1.5	Series registered on position create
M1-06	Devices: enrol, assign with counter transfer and drained check, suspend/retire; gateway revoke event	1.5	Scenario 6.6
M1-07	Users and credentials: handlers with the provider wrapper; PIN policy; deactivation guard	2	Create, reset, deactivate against dev realm
M1-08	Roles, assignments, SoD pairs: handlers with guardrails; template diff; property tests	2	Scenario 6.3; property tests
M1-09	External grants; scope resolution for EXTERNAL; expiry job registration	0.5	Scenario 6.5
M1-10	Snapshot contributors and change-log producers; till snapshot verified by the till simulator	1	Operator snapshot round trip
M1-11	BulkRegister with validation report; performance test	1.5	40 shops under 60 s
M1-12	Web screens (nine) on the shell; Playwright flows	3	Both Playwright flows green
M1-13	Contract freeze: publish m1party.yaml v1, api package review with the M2 developer, doc 33 slice registered	0.5	M2 acknowledges

Total about 19.5 developer-days – about four weeks for one developer; the contract freeze (M1-13) is targeted for the end of the second week so M2 can start against api while screens finish.

11. Open points: what to assume, whom to ask

Point	Assume	Decider	If different
Entity activation prerequisites (doc 21 DR-6)	Officer named, admin user, VAT registration	Federation admins	Adjust ActivateEntity guards; checklist tile
Direct Federation→MPCS trading (DR-3)	Flag present, default off	Federation admins	Flip the config item; no code change
External grant maximum (DR-4)	12 months	Federation, DCD liaison	Change the CHECK constraint and validator
Returning staff (DR-5)	New user with succeeds_user_id	Federation admins	Add a Reactivate transition
Responsible officer fields (DR-1) and sod_pair (DR-2)	Adopted per doc 18 v0.2	Closed	–
MPCS admin reading shop users' rows	RLS policy variant allowing home-entity-or-parent for shop-scoped users (19A §1)	Platform pair	Policy text in your migration changes; handlers unchanged
Price-list validation at ActivateRelationship	M3 query interface stubbed until doc 23; stub returns true for the seed list	M3 developer	Replace the stub; tests already assert the call

End of document. 21A v0.1; the repository's `m1party/README.md` and its tests become the living version once M1 is merged.